

Topics :

- Modules
- Managing Several Values
 - Lists
 - Tuples

1

1

Modules

- A Haskell program is a collection of **modules**
- The **main module will load up the other modules** and then **use the functions** defined in them to do something
- Each module
 - has a **unique** name
 - contains a **collection of related definition, types and type classes**
 - **may import** definitions of other modules

2

2

- Modules provide a way to **structure your code in a modular and organized way**, making it **easier to maintain and reuse code**
- They also make it **easier to share code with others**, as you can package your functions into a module and distribute it for others to use

3

3

- Having code split up into several modules has **several advantages**:
 - if a module is generic enough, the functions it exports can be **used for many different programming tasks**
 - if your own code is separated into self-contained modules which don't rely on each other too much (we also say they are loosely coupled), you can **reuse (some of) them separately later on**
 - splitting code into several parts, each of which has a specific purpose, makes writing code **more manageable**

4

4

Naming Modules

- Module names are **alphanumeric**
- Module names must **begin with an uppercase letter**
- There is **no formal connection** between a Haskell module and the file system that would (typically) support it
- We will **adopt the following conventions**
 - there is **one module per file**
 - the file **name is the name of the module it contains**

5

5

Define a Module

- To define a module, use the command `module ModuleName where`

```
module MyModule where  --define a module called MyModule
function1 x y = x + y
function2 x y = x * y
```

- The module is called `MyModule` and has two functions `function1` and `function2`
- The `where` clause **specifies the implementation of the functions**
- To use this module in another module or script, you should **import** it

6

6

▪ Every module in a Haskell program must be closed

- This means that every name explicitly mentioned by the source code must be either defined locally or imported from another module
- However, entities that the compiler requires for type checking or other compile time analysis need not be imported if they are not mentioned by name
- That is, the import of a variable `x` does not require that the datatypes and classes in the signature of `x` be brought into the module along with `x` unless these entities are referenced by name in the user program
- The Haskell system silently imports any information that must accompany an entity for type checking or any other purposes

7

7

Import a Module

- The syntax for importing modules in a Haskell script is


```
import <module_name>
```
- `import MyModule` will import everything exported from `MyModule`
- Importing a module must be done before defining any functions, so imports are usually done at the top of the file
- To import several modules in a script, each import statement should be on a separate line

```
import <moduleName1>
import <moduleName2>
import <moduleName3>
```

```
...
```

8

8

- To import a selection of functions from a module, you can **selectively import just those functions**
- For example, to import only `function1` from `MyModule`, just use

```
import MyModule (function1)
```
- To import several functions, **separate them with commas in the () brackets**

9

9

▪ Example:

```
import Data.Char(toLower, toUpper) -- import only toLower and toUpper
from Data.Char
import Data.List -- import everything exported from Data.List
import MyModule -- import everything exported from MyModule
```

10

10

- To import all of the functions of a module **except a few select ones** use the keyword `hiding`
- For example, to import all the functions from `Data.List` except the `nub` function

```
import Data.List hiding(nub)
```

- Whatever follows the hiding keyword **will not be imported**
- To hide multiple items, listing them with `()` and comma-separation
- This is often useful when several modules export **functions with the same name** and we want to avoid name conflicts

11

11

- Algebraic **datatypes** and type **synonyms** cannot be hidden
- These are **always imported**
- If you have a datatype defined in multiple imported modules, you must use **qualified names**
- With qualified imports, all the imported values include the module names as a prefix
- For example, assuming a function `functionName` is in two different modules but performing different tasks

```
MyModule.functionName variables -- uses the function from MyModule
```

```
MyOtherModule.functionName variable -- use the function from MyOtherModule
```

12

12

- Imported modules may be renamed and assigned a local alias in the importing module using the as clause

- For example

```
import qualified MyVeryLongModuleName as ModAlias
```

- This allows us to use ModAlias instead of MyVeryLongModuleName as prefix for the imported functions
- This means entities must be referenced using `ModAlias.` as a qualifier instead of `VeryLongModuleName.`
- This renaming works with both qualified and regular importing

13

13

- As long as there are no conflicting items, multiple modules can be imported and renamed the same

- Example

```
import MyLongModule as My
import MyCompletelyDifferentModule as My
```

- In this case, both the functions in MyLongModule and the functions in MyCompletelyDifferentModule can be prefixed with My

14

14

Haskell Modules

- The Haskell standard library is split into modules
- Each of them contains functions and types that are somehow related and serve a common purpose

15

15

▪ List Module

- List provides some functions to work with list type data
- Once imported, the List module provides a wide range of functions

16

16



▪ Char Module

- The Char module has plenty of predefined functions to work with the **Character** type

17

17



▪ Map Module

- Map is an **unsorted value-added pair type data type**
- It is a widely used module with many useful functions
- .

18

18

▪ Set Module

- The Set module has some very useful predefined functions to manipulate mathematical data
- A set is implemented as a binary tree, so all the elements in a set must be unique

19

19

❖ Example

- Create a custom module that can be called at other programs
- To implement this custom module, we will create a separate file called myCalculator.hs along with our main.hs.
- Let us create the custom module and define a few functions in it.

20

20

```
module MyCalculator where
```

```
addTwo::Int->Int->Int --adds the two numbers together
```

```
addTwo x y = x+y
```

```
averageTwo:: Float -> Float -> Float --calculates the average of two floats
```

```
averageTwo a b = (a + b)/2.0
```

```
averageThree::Float->Float->Float->Float --calculates average of 3 numbers
```

```
averageThree a b c = (a+b+c)/3.0
```

21

21

```
areSorted::Int->Int->Int->Bool --returns True if the three numbers are arranged
```

```
areSorted x y z
```

```
    | (x>y) && (y>z) = True
```

```
    | (x<y) && (y<z) = True
```

```
    | otherwise      = False
```

```
maxThree::Int->Int->Int->Int --returns the largest of the three integers
```

```
maxThree x y z
```

```
    | x>=y && x>=z = x
```

```
    | y>=z         = y
```

```
    | otherwise    = z
```

22

22

```
showEven::Int-> Bool  --returns True if the number is even
showEven x = do
  if rem x 2 == 0
    then True
    else False
```

23

23

- To test the module
 - Ensure you **are in the same directory** where your module is saved
 - Then, **start GHCi** `C:\CSI243>ghci`
 - **Load the program file** by typing `Prelude>:load MyCalculator`
 - Remember that you can abbreviate the command `:load` to just `:l`
 - You can also load the module by just using `C:\CSI243>ghci MyCalculator`
 - If properly compiled, the prompt should be `*MyCalculator>`
 - Test the functions

25

25

- To import the module into a program, we need to create the main.hs program

```
import MyCalculator

main = do
    print("Hello There")
    print("Let us calculate!!!")
```

26

26

- Using `ghci` `mainModule` will import the `MyCalculator` module and compile it

```
GHCI, version 8.10.7: https://www.haskell.org/ghc/  :? for help
[1 of 2]Compiling MyCalculator      (MyCalculator.hs, interpreted )
[2 of 2]Compiling Main              (mainModule.hs, interpreted )
Ok, two modules loaded.
*Main>
```

- To use the functions, just type `main` at the prompt

27

27

- To use the functions in `MyCalculator`, just type `main` at the prompt

```
*Main> main
"Hello There"
"Let us calculate!!!"
*Main> averageTwo 24 67
45.5
*Main> addTwo 89 23
112
*Main>
```

28

28

Lists

- A list contains **several values**
- The values are
 - enclosed in **square brackets []**
 - separated by **commas**
 - all of the **same type**

example: `[1,5,8]` is a list with three vales, 1, 5 and 8

`[10..20]` is a list of eleven integers from 10 to 20

29

29

Lists are homogenous!!

- Lists are a **homogeneous data structure** i.e., **all the elements** in a list are of **the same type**

- This means that we can **have a list of integers** or a **list of characters**, but we can't have a list that has a few integers and then a few characters

- Trying to define a list of mixed type vales will results in an error message

```
Prelude>let numbers = [1,2,3,4,5] will build a list of integers 1 to 5
Prelude>let truths = [True, False, True]
Prelude>let strings = ["yes", "they", "are"]
Prelude>characters = ['a','b','c']
```

- An **empty** list is written as []
- A **singleton list** or simply singleton is a list of length 1

30

30

Basic List Functions

- `null lst` - returns **True** if the list `lst` is empty, **False** otherwise

`null [5,4,3,2,1]` will return **False** because the list is not empty

- `head lst` - returns the **first value of lst**

`head [5,4,3,2,1]` will return **5**

- `tail lst` - returns the **list of values in lst, leaving out the head**

`tail [5,4,3,2,1]` will return **[4,3,2,1]**

31

31

- `last lst` - returns the **last value in lst**

`last [5,4,3,2,1]` will return 1

- `init lst` - returns the **list of values in lst, without the last element** (opposite of tail)

`init [5,4,3,2,1]` returns `[5,4,3,2]`

- `v::lst` - cons, returns the **values in lst with v added at the head**

`6:[5,4,3,2,1]` returns `[6,5,4,3,2,1]`

32

32

- `take v [lst]` - returns **the first v elements** in lst

- if lst has less than v elements, returns lst

```
Prelude>let numbers = [1,2,3,4,5]
```

```
Prelude>take 2 numbers
```

```
[1,2]
```

```
Prelude>take 0 numbers
```

```
[]
```

```
Prelude>take 6 numbers
```

```
[1,2,3,4,5]
```

33

33

o `length lst` - returns the **number of values in lst**
`length [5,4,3,2,1]` returns 5

o `lst!!k` - returns the ***k*th value** in `lst`, indexed from 0

```
Prelude> let numbers = [1,2,3,4,5]
Prelude> numbers!!1
2
Prelude> numbers!!0
1
Prelude> numbers!!9
*** Exception: Prelude.!!: index too large
```

34

34

o `reverse lst` - **reverses the order** of elements in `lst`

```
Prelude> let numbers = [1,2,3,4,5]
Prelude> reverse numbers
[5,4,3,2,1]
```

o `sum lst` - **adds up all** the elements in `lst`

o `product lst` - returns the **product** of all the elements in `lst`

o `minimum lst` - returns the **smallest number** in `lst`

o `maximum lst` - returns the **largest number** in `lst`

35

35

- o `lst0++lst1` - **concatenates** `lst0` and `lst1`, starting with all elements of `lst0` followed by all elements of `lst1`

```
Prelude>let lst1 =[1,2,3,4]
Prelude>let lst2 =[4,5,6,7]
Prelude>let lst3 =[8,9,10]
Prelude>lst1++lst2
[1,2,3,4,4,5,6,7]
Prelude>lst2++lst1
[4,5,6,7,1,2,3,4]
Prelude>lst1++lst2++lst3
[1,2,3,4,4,5,6,7,8,9,10]
```

36

36

Cons a List

- The `:` operator can be **used to cons (add) elements to a list**
- The process is typically called **consing**
- Consing an element to a list **returns the list with the element as the first element in the new list**
- You can cons as **many times as needed**
- **You can not cons a list on to an element**

```
Prelude>let numbers = [1,2,3,4]
Prelude>0:numbers
[0,1,2,3,4]
Prelude> 1:2:3:numbers
[1,2,3,1,2,3,4]
Prelude>
```

37

37

A list of lists

- You can have **lists of lists**
- The lists within a list **can be of different lengths**
- But they **all have to be of the same type**
- e.g. `[[1,2], [3,4],[5,6]]` -- list of same type
`[[1,2],[2,3,4],[5]]` -- still a valid list

38

38

- To create a list at the prompt, we can use the `let` keyword

```
Prelude>let intList = [1,2,3,4,5]
Prelude>intList
[1,2,3,4,5]
Prelude>let charList = ['a','b','c']
Prelude>charList
"abc"
Prelude>let stringList = ["Tebogo", "Lerato", "Bontle"]
Prelude>stringList
["Tebogo","Lerato","Bontle"]
```

39

39

- Creating a list that has elements of different types will result in an error

```
Prelude>let wrongList = [1,'a',"string"]

<interactive>:12:24: error:
  *Couldn't match expected type `Char' with actual type `[Char]'
  * In the expression: "string"
    In the expression: [1, 'a', "string"]
    In an equation for `wrongList': wrongList = [1, 'a', "string"]
Prelude>
```

40

40

Tuples

- A tuple is a **finite sequence of possibly different types**
- Components are **separated by commas** and enclosed in **round brackets ()**
- Tuples have a **fixed number of things (immutable)**, you can not cons to a tuple
- The elements in a tuple do not need to be all of the same type(**heterogenous**)
- Example:


```
(50, "csi243", "2013/14")
('2', False)
```

41

41

Tuples are immutable

- Tuples are **immutable**
- The elements in a tuple are fixed
- You can not add elements to a tuple as the program runs
- Therefore, tuples are typically used when you know in advance how many values are to be stored

42

42

Tuple Arity

- **Arity** gives the number of components in a tuple
- A unit is an empty tuple (), that is of arity 0
- An n-tuple is a tuple with n elements
- A pair is a tuple of arity 2
- A triple has arity 3
- Tuples of arity 1 not allowed

43

43

Tuples

- There are **no restrictions on the types of the components of a tuple**, may include:-
 - tuples of tuples
 - tuples of lists
 - lists of tuples

- **Examples:-**

`(50, ("cs", False) )` - tuple of tuples

`([ '2', 'a', 'x'], [False, False, False ])` - tuple of lists

`[ ( "cs", False), ( "sc", True ) ]` - list of tuples

44

44

- Most of the time tuples are in pairs, therefore two basic methods used:

- **fst** which gives the first element of a tuple of arity 2

`fst(5, 'h')` returns 5

`fst("Tebogo", "Lesego")` returns "Tebogo"

- **snd** which gives the second element of a pair (tuple)

`snd(5, 'h')` returns 'h'

`snd("Tebogo", "Lesego")` returns "Lesego"

45

45

Uses of Tuples

- Tuples can be used to return multiple values from a function
 - Example: `addAndSub :: Int -> Int -> (Int, Int)`
takes two integers and returns two integers (tuple) - first int returned is sum, second int returned is difference
- Tuples can be used as function arguments
 - Example: `add :: (Integer, Integer) -> Integer`
 - Example: `swap :: ( a, b ) -> ( b, a )`

47

47

List Comprehension

- A distinct feature of Haskell
- Allows a **list to be created based on another list**
- That is, **describe a list in terms of another list**
- The syntax consists of two parts separated by a pipe |
`[first part | second part]`

49

49

❖First Part

- The first part **before the pipe |**
- The **output function**
- Specifies the **function to apply to the input elements**
- For each symbol value that satisfies the predicate, the output function is evaluated and its value included in the constructed list

50

50

❖Second Part

- Appears **after the pipe |**
- Specifies **how to get the input elements**
- Can either be a
 - Generator
 - Test(s)

51

51

❖ Generator

- Input list in the form `symbol<-listExpression` or a pattern
 - Generates the data from which the results are built
- e.g. `x<-[1..10]`
- The symbol `x` should take each value from the `listExpression` `[1..10]`, that is, `x` is an integer between 1 and 10
- Multiple generators allow elements from two or more lists to be combined

52

52

❖ Test(s)

- These one or more comma separated predicates/conditions
 - Boolean expressions that filter only those values that are true
- e.g. `x<-[1..10], mod x 2 ==0`
- for each value between 1 and 10, only those values that are even will be used

53

53

1. $[x^2 | x \in \{1, 2, 3, 4, 5\}]$
2. $[x^2 | x \in [1..5], \text{mod } x \ 2 == 0]$
3. $[x^2 | x \in [1..10], x^2 \geq 12]$
4. $[x | x \in [50..100], \text{mod } x \ 7 == 3]$
5. $[x+y | x \in \{1, 2, 3\}, y \in \{-3, 0, 3\}]$

54